CSA0619 CO4 AT1 ANSWERS

By Antony Magrin J(192572103)

Q3. Traveling Salesman Problem (TSP)

Solved using Dynamic Programming — Held-Karp Algorithm

1. Problem Understanding

Problem restated:

- Given n cities and an n×n cost matrix Cost[i][j], find a tour that starts at a fixed city, visits every other city exactly once, and returns to the start, minimizing total cost.
- This is the classic symmetric/asymmetric TSP — an NP-hard combinatorial optimization problem.

Constraints identified:

- n can be as small as 1 (trivial tour, cost 0) — must not crash on edge inputs.
- Cost matrix is not guaranteed symmetric (Cost[i][j] may differ from Cost[j][i]); diagonal Cost[i][i] = 0.
- A full Hamiltonian cycle is required — no city is skipped and none is repeated except the return to start.
- For competitive settings, n is typically bounded (n ≤ ~18–20) because the problem is NP-hard; brute force is only viable for very small n.

Edge cases considered:

- n = 1: only the starting city exists, minimum cost = 0.
- n = 2: only one possible round trip; cost = Cost[0][1] + Cost[1][0].
- Disconnected graph / unreachable city: represented using ∞ (INT_MAX) so that path is never chosen.
- Asymmetric costs: since Held-Karp works on directed transitions Cost[i][j], asymmetry is handled correctly without modification.

2. Algorithm Used

Why Held-Karp DP instead of brute force:

Approach	Time Complexity	Space Complexity	Max Practical n
Brute Force (permutations)	O(n!)	O(n)	~10
Held-Karp (DP + bitmask)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	~20
Approximation (Nearest Neighbor)	$O(n^2)$	O(n)	large n, no optimality guarantee

Held-Karp reduces the search space using bitmask DP over subsets of visited cities, avoiding redundant recomputation of overlapping subproblems (optimal substructure + overlapping subproblems — the two DP prerequisites are both satisfied here).

2.1 State Definition

dp[S][i] = minimum cost to visit exactly the set of cities S, ending at city i, having started from city 0.

2.2 Base Case

```
dp[{0}][0] = 0
```

2.3 Transition

For every subset S containing city 0 and current city i, and every unvisited city j:

```
dp[S ∪ {j}][j] = min( dp[S ∪ {j}][j], dp[S][i] + Cost[i][j] )
```

2.4 Final Answer

```
Minimum Cost = min over i ≠ 0 of ( dp[FULL_SET][i] + Cost[i][0] )
```

2.5 Pseudocode

```
function HeldKarp(Cost, n):
   dp = array[2^n][n], all = INFINITY
   dp[1][0] = 0                    // mask with only city 0 set

   for mask = 1 to (2^n - 1):
      for i = 0 to n-1:
         if bit i not in mask: continue
         if dp[mask][i] == INFINITY: continue
         for j = 0 to n-1:
            if bit j in mask: continue
            newMask = mask | (1 << j)
            dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + Cost[i][j])

   fullMask = (1 << n) - 1
   answer = min( dp[fullMask][i] + Cost[i][0] )  for i = 1..n-1
   return answer
```

3. Worked Example (Sample Input Walkthrough)

Input cost matrix (n = 4):

	C0	C1	C2	C3
C0	0	10	15	20
C1	10	0	35	25
C2	15	35	0	30
C3	20	25	30	0

DP table trace (key states, mask = subset of visited cities including city 0):

Subset (mask)	Ending City	dp value	Formed From
{0}	0	0	base case
{0,1}	1	10	dp[{0}][0] + C(0,1)
{0,2}	2	15	dp[{0}][0] + C(0,2)
{0,3}	3	20	dp[{0}][0] + C(0,3)
{0,1,2}	2	45	dp[{0,1}][1] + C(1,2)
{0,1,3}	3	35	dp[{0,1}][1] + C(1,3)
{0,2,3}	3	45	dp[{0,2}][2] + C(2,3)
{0,1,2}	1	50	dp[{0,2}][2] + C(2,1)
{0,1,3}	1	45	dp[{0,3}][3] + C(3,1)
{0,2,3}	2	50	dp[{0,3}][3] + C(3,2)
{0,1,2,3}	1	70	min(85, 70) via city 3
{0,1,2,3}	2	65	min(80, 65) via city 3
{0,1,2,3}	3	75	min(75, 75)

Closing the tour (adding return edge to city 0):

dp[{0,1,2,3}][1] + Cost[1][0] = 70 + 10 = 80
dp[{0,1,2,3}][2] + Cost[2][0] = 65 + 15 = 80
dp[{0,1,2,3}][3] + Cost[3][0] = 75 + 20 = 95

Minimum Cost = 80

Optimal route: 0 → 1 → 3 → 2 → 0 (cost 10 + 25 + 30 + 15 = 80)

4. Time and Space Complexity

Time and space bound:

- Time: $O(n^2 \cdot 2^n)$ — for n = 18, that is roughly $18^2 \times 2^{18} \approx 8.5 \times 10^7$ operations, feasible within a 1–2 second limit.
- Space: $O(n \cdot 2^n)$ integers — for n = 20 this is $\sim 20 \times 10^6 \approx 20$ MB (using 4-byte ints), still within typical memory limits.

Scalability guidance:

- $n \leq 20$: Held-Karp DP is the right choice — exact optimum, feasible time/memory.
- $n > 20–25$: exact DP becomes infeasible (2^n explodes); switch to heuristics/metaheuristics such as Nearest Neighbor + 2-opt, Christofides (metric TSP), Genetic Algorithms, or Branch-and-Bound with pruning for near-optimal answers in competitive time limits.
- I/O and constant-factor optimizations: use iterative bitmask loops (not recursion) to avoid function-call overhead and stack depth issues, and use fast I/O (scanf/printf or buffered input) for large test files.

5. Test Cases

Test cases covering functional correctness, boundaries, and stress conditions:

#	Description	Input (n, matrix)	Expected Output
1	Given sample (functional test)	n=4, sample matrix	Minimum Cost = 80
2	Smallest valid input	n=1, single city	Minimum Cost = 0
3	Two cities	n=2, C(0,1)=C(1,0)=5	Minimum Cost = 10
4	Symmetric vs asymmetric cost	n=4, Cost[i][j] ≠ Cost[j][i]	Correct directed-cost route chosen
5	All equal costs	n=5, every edge = 10	Minimum Cost = 50 (n × edge cost)
6	Large sparse-like input (stress)	n=15–18, random costs	Completes within time limit; matches brute force on n≤10
7	Disconnected / invalid edge (INF cost)	one edge = ∞ (unreachable)	Algorithm avoids that edge; still finds valid tour if one exists

Validation strategy:

- Cross-check DP output against brute-force permutation search for $n \leq 10$ to confirm correctness.
- Test with n = 1 and n = 2 to confirm no out-of-bounds bitmask access.

- Test with asymmetric matrices to confirm directional costs are respected (Cost[i][j] used, not Cost[j][i]).
- Stress-test with n = 18–20 and randomized costs to confirm the solution completes within the time limit.

6. Conclusion

The Held-Karp Dynamic Programming approach solves the sample instance with Minimum Cost = 80, matching the expected output, via the optimal route $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$. It improves on brute force's O(n!) to $O(n^2 \cdot 2^n)$, correctly handles asymmetric costs and edge cases (n=1, n=2, unreachable edges), and remains practical for competitive-programming-scale inputs ($n \leq \sim 20$).